NativeAOT

Release-only. NativeAOT publish and the AOT suites are release-time steps. During alpha, do not run `just publish`, `just aot-test`, `just aot-smoke`, or `just release`; those run only when the owner calls a major public release. Scale publishes as a self-contained native binary with .NET NativeAOT. The AOT artifact is the product binary; the test suite is a separate JIT-only deployable (`projects/scale-tests`, D176). Publish and run the product:

```
dotnet publish bases/scale-cli/scale.fsproj -c Release -r linux-x64 -o artifacts/
native/linux-x64
./artifacts/native/linux-x64/scale
```

`just publish linux-x64` and `just aot-test linux-x64` wrap the publish and the product smoke; `just aot-test` never tests a stale artifact, and it does not run the suite because the suite is not in this artifact. `just release` is the local release gate: it runs `just check` (the JIT suite in `projects/scale-tests`), publishes the host RID, runs the full product from an isolated temporary directory, drops the Windows-only `libz3.dll`, and writes `artifacts/scale-<rid>.tar.gz`. It has no git or network side effects. The GitHub release workflow publishes a native product binary on every published release for `linux-x64`, `win-x64`, and `osx-x64`, and attaches the archives to the release; it runs the complete gate — formatting, build, and the JIT suites — before it publishes anything, and the native smoke test runs the full product binary (D27, D176).

Every recipe that writes or deletes an output directory takes its runtime identifier from a closed allowlist (`just check_rid <rid>`), and the identifier must be a plain name: it reaches the filesystem through the output path, so an unvalidated one could name a directory outside the repository.

`dotnet publish` copies `components/demo/demo/generated` and `witnesses` beside the binary, so the publish output itself runs standalone. `just aot-test`, `just release`, and CI copy that output to a temporary directory outside the repository before running the product smoke, which catches a missing payload. The demo looks for `bases/scale-cli/scale.fsproj` above the executable first, then falls back to the executable's directory.

JIT-only suites

The test suite is a separate deployable (`projects/scale-tests`, D176); the AOT product has no `--tests`, `--all-tests`, or `--granular-tests` modes. The granular runner is JIT-only: `granular.Logging` is not NativeAOT-compatible (reproduced with granular 11.1.1 alone; `TrimmerRootAssembly` does not help). The product publish payload does not copy the `snapshots/` baselines. See D23 for the original suite-in-deployable decision; D176 supersedes its product coupling.

Kernel AOT gate

A kernel change is validated with `just kernel-aot linux-x64` (D180). It publishes the kernel as a NativeAOT shared library and runs the C conformance harness against it: version equality, image schema refusal, multi-handle lifetime, primitive dispatch, and the freed-handle refusal. This is the one NativeAOT step allowed in alpha; the product publish remains release-only. The `Kernel AOT` CI workflow runs the same gate on changes under `components/kernel/**` and `components/adapter-c/**`, and the release workflow runs it before publishing.

Compatibility rules

The project sets these properties in `bases/scale-cli/scale.fsproj`:

- `PublishAot` — build native code on `dotnet publish`.

- `IlcGenerateCompleteTypeMetadata` and `IlcTrimMetadata=false` — keep F# type metadata. Without both, reflection over an application union throws `KeyNotFoundException` under AOT.
- `StripSymbols` and `DebugType=none` in `Release` — do not ship a symbol file.

Four rules keep the code AOT-safe:

1. **Do not use numeric `printf` specifiers or `%A`.** F# specializes `%d`, `%g`, `%f`, `%b` and friends at runtime through `MethodInfo.MakeGenericMethod`, which NativeAOT cannot execute. `%A` is also unreliable: on a union it drops the field values (`UnknownWord "x"` prints as `UnknownWord`), and on a primitive it can still throw `NotSupportedException` (`ObjectPrinter.GenericToString<T>`). Use string interpolation, `.ToString(...)`, or a named formatter such as `Types.Core.describeError`. `%s` is safe.
2. **Do not serialize F# records with `System.Text.Json` reflection.** The source-generation path is not usable from F# either, and the reflection path throws under AOT. `repair.fs` writes its bundle and corpus with `Utf8JsonWriter`, which is AOT-safe.
3. **Do not render a Hedgehog failure report.** `Property.checkBool` renders `after %d tests` with a numeric printf specialization, which throws under AOT. `refiner.fs` reads the shrunk counterexample from the `Report`'s `Journal` and serializes the recheck data directly.
4. **Z3 is a native dependency.** `Microsoft.Z3` ships `libz3.dll` for Windows and `libz3.dylib` for macOS x64; the publish output for those RIDs includes it. On Linux the managed wrapper P/Invokes the system `libz3.so`, so install it (`libz3-dev` or `libz3-4` on Debian/Ubuntu, `z3` on Arch). The AOT binary is not fully self-contained on Linux until that shared library is present.

`FSharp.Core`, `Hedgehog`, `granular`, `Verify`, `SimpleInfoName`, and `EmptyFiles` still produce dependency trim/AOT warnings (`IL3053`, `IL2104`) in the test deployable. They are not warnings from the product code, and the published product does not include the test suite (D176).

No `sprintf` on error paths exercised by AOT. The corrupt-store reader builds its messages by string concatenation. A `sprintf` with a runtime argument can require `MakeGenericMethod`, which NativeAOT does not support; it surfaced as `NotSupportedException` in the corrupt-store test. Prefer `string` and concatenation on paths the native suite exercises.

The Reflection.Emit exception. A language-to-.NET backend may use `System.Reflection.Emit` and `PersistedAssemblyBuilder`, but only as a JIT-only build tool. It must be unreachable from the AOT entry point and excluded from the native publish (a separate project, or a conditional `Compile` item plus a stub that returns a named refusal). A runtime `RuntimeFeature.IsDynamicCodeSupported` check alone is not enough: the AOT analyzer still sees the call and the method is still compiled. The generated assembly is produced before publish and statically referenced by the AOT build, so the native binary never emits code at run time.

The AOT product smoke also exercises the language entry points in the isolated payload: it runs a `.prose` program, runs a spell's `main`, and casts a word from a book. The smoke asserts the prose result, the spell classification, and the cast result, so a missing book/prose payload or a broken CLI selector fails the release artifact test.